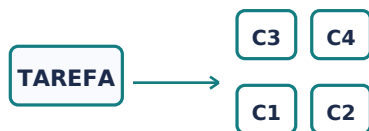


PARALELISMO & ARQUITETURAS



$$S = T_{\text{serial}} / T_{\text{paralelo}}$$

Executar múltiplas operações ao mesmo tempo para aumentar desempenho e throughput.

- **Paralelismo:** tarefas simultâneas em múltiplos recursos
- **Concorrência:** tarefas avançam de forma intercalada ou simultânea
- **Flynn:** SISD, SIMD, MISD e MIMD
- **SIMD:** mesma instrução em vários dados; comum em GPU
- **MIMD:** processadores executam instruções distintas
- **Speedup:** $S = T_{\text{serial}} / T_{\text{paralelo}}$
- **Eficiência:** $E = S/p$, com p processadores
- **Lei de Amdahl:** a parte serial limita o speedup máximo

Dica: Mais núcleos não garantem ganho linear. Sempre identifique a parte serial e o custo de sincronização/comunicação.

MEMÓRIA COMPARTILHADA & SINCRONIZAÇÃO



Coordenar threads que acessam dados comuns sem produzir resultados inconsistentes.

- **Threads:** compartilham memória do processo
- **Race condition:** resultado depende da ordem
- **Seção crítica:** acessa recurso compartilhado
- **Mutex:** exclusão mútua para um recurso
- **Semáforo:** controle por contagem/sinal
- **Barreira:** sincroniza fases das threads
- **Coerência de cache:** mantém cópias consistentes
- **False sharing:** disputa indevida de linha de cache

Dica: Evite compartilhar estado quando não for necessário. Menos sincronização costuma significar menos bloqueio e melhor escalabilidade.

SISTEMAS DISTRIBUÍDOS & COMUNICAÇÃO



Coordenar computadores independentes que cooperam por rede como um único sistema lógico.

- **Nó:** máquina/processo participante
- **Mensagem:** comunicação entre nós
- **Latência:** atraso da mensagem; **largura:** taxa de dados
- **RPC:** chamada remota como chamada local
- **Cliente-servidor:** solicita ↔ oferece serviço
- **P2P:** nós atuam como cliente e servidor
- **Relógios:** não há tempo global perfeito
- **Falhas:** rede e nós falham

Dica: Em sistemas distribuídos, a rede faz parte do problema: mensagens atrasam, podem falhar e não existe memória global compartilhada.

CONSISTÊNCIA, FALHAS & ESCALABILIDADE



Projetar serviços distribuídos que continuem corretos e disponíveis mesmo sob carga e falhas.

- **Replicação:** cópias para disponibilidade/leitura
- **Particionamento:** divide dados/carga entre nós
- **Consistência forte:** leitura vê valor recente
- **Consistência eventual:** réplicas convergem
- **CAP:** sob partição: consistência x disponibilidade
- **Tolerância a falhas:** detectar, isolar, recuperar
- **Balanceamento:** distribui requisições

Dica: Antes de escolher consistência forte ou eventual, pense no requisito do sistema: dinheiro e estoque pedem garantias diferentes de feed e cache.